

SK네트웍스 Family AI 과정 23기

데이터 전처리 학습된 인공지능 모델

withDOG (윳독) — 반려견 동반 가능 장소 추천 및 고객 상담 챗봇

산출물 단계	데이터 전처리
평가 산출물	학습된 인공지능 모델
제출 일자	2026-04-23
깃허브 경로	https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN23-FINAL-3Team

주력 모델: KoELECTRA-base-v3 Fine-tuned · 부가 모델: YOLOv8 Nano (Inference only)

1. 모델 목적

본 모델은 withDOG 챗봇 서비스에 탑재되는 한국어 인텐트 분류기이다. 사용자가 입력한 자연어 발화를 4개 인텐트(장소추천 / 다이어리 작성 / 시설정보 / 기타)로 분류하여, 발화의 목적에 맞는 후속 AI 처리 체인(RAG 기반 장소 추천, 6하원칙 다이어리 생성, 장소 상세 정보 조회, 일반 Fallback 응답)으로 라우팅하는 것을 목적으로 한다.

분류기가 LLM보다 선행하여 동작함으로써 (1) 불필요한 LLM 호출을 줄여 비용을 최적화하고, (2) 체인별 전문 프롬프트·도구를 사전에 배치하여 응답 품질을 안정화하며, (3) 범위 밖 질의(OTHER 클래스)를 조기 차단하여 할루시네이션 리스크를 낮춘다.

1.1 부가 모델 목적

주력 모델 외에 YOLOv8 Nano가 이미지 파이프라인에 탑재되어 있다. AI로 생성된 반려견 다이어리 삽화에서 강아지 품종 객체를 감지·분류하여, 사용자 반려견과 생성 이미지 간 시각적 일관성을 보조 검증하는 용도로 사용된다. 별도 fine-tuning 없이 Ultralytics 공식 체크포인트를 추론 전용으로 활용한다.

2. 모델 아키텍처 설계

2.1 선정 모델

Base Model: monologg/koelectra-base-v3-discriminator (HuggingFace Hub 공개 모델)

Fine-tuning Task: Sequence Classification (ElectraForSequenceClassification, 4-class 분류 헤드)

KoELECTRA v3는 한국어 위키·나무위키·뉴스·모두의말뭉치 등 약 34GB 코퍼스로 사전학습된 한국어 특화 ELECTRA 모델이다. Google ELECTRA 구조를 따르며, Base 기준 약 110M 파라미터를 보유한다.

2.2 아키텍처 개요

입력 발화는 WordPiece 토큰라이저를 거쳐 토큰 ID 시퀀스가 되고, 사전학습된 KoELECTRA Encoder를 통과한 뒤 [CLS] 토큰의 최종 은닉 상태가 분류 헤드에 입력되어 4개 클래스 로짓을 산출한다. 로짓은 후처리 단계에서 Softmax를 통해 확률로 변환되며, argmax로 최종 인텐트가 결정된다.

계층	구성 요소	역할
입력층 (Tokenizer)	ElectraTokenizer (WordPiece, Vocab 35,000)	한국어 문장을 서브워드 토큰 ID 시퀀스로 변환
임베딩층	Token Emb. + Segment Emb. + Position Emb. (Hidden 768)	토큰·세그먼트·위치 정보를 768차원 벡터로 합성
인코더(Transformer Block × 12)	Multi-Head Attention (12 heads) + FFN + LayerNorm + Residual	문맥 기반 의미 표현 학습 (사전학습 가중치 유지)
Pooling	[CLS] 토큰 최종 은닉 상태	문장 전체를 단일 768차원 벡터로 요약
출력층 (분류 헤드)	Dropout(0.1) → Linear(768→4) → Softmax	4개 인텐트 클래스에 대한 확률 분포 산출

2.3 Discriminator 기반 아키텍처의 특징

ELECTRA는 Generator와 Discriminator 두 네트워크로 구성되는 사전학습 프레임워크로, Masked Language Modeling(MLM)을 대체하는 Replaced Token Detection(RTD) 태스크를 사용한다. Generator가 일부 토큰을 그럴듯한 대체어로 치환하면, Discriminator는 각 토큰이 원본인지 치환된 것인지 이진 분류한다. 본 프로젝트는 사전학습된 Discriminator만 가져와 Fine-tuning에 사용한다.

MLM 기반 BERT 대비 장점은 다음과 같다.

- **학습 신호 밀도:** MLM은 전체 토큰의 15%(마스킹된 토큰)에서만 loss 신호가 발생하는 반면, RTD는 모든 토큰에서 이진 판별 loss가 발생 → 동일 FLOPs로 약 4~10배의 학습 신호 확보
- **사전학습 효율:** 동일 파라미터·동일 컴퓨팅 예산에서 ELECTRA-Base가 BERT-Base 대비 GLUE/KLUE 평균 점수 우위

- **소규모 Fine-tuning 강건성:** RTD가 모든 토큰 표현을 강제로 학습시켜 사전학습 표현이 더 균일하게 품질이 높음 → 수백 건 규모의 다운스트림 fine-tuning에서 일관된 수렴
- **추론 비용 동일:** 추론 시에는 Discriminator만 사용되므로 BERT-Base와 동일한 latency·메모리 footprint 유지

2.4 파라미터 수 및 주요 사양

항목	값
총 파라미터 수	약 110M (Base 기준)
Hidden Size	768
Attention Heads	12
Hidden Layers (Transformer Block)	12
Intermediate Size (FFN)	3,072
Vocab Size	35,000 (WordPiece)
Max Position Embeddings	512
Input Max Length (본 프로젝트 학습 기준)	128 tokens
Activation	GELU
Classification Head	Linear(768 → 4) + Softmax

2.5 설계 근거

KoELECTRA 기반 fine-tuning 구조는 (1) 사전학습 모델의 가중치를 재사용하여 490건 규모의 소량 데이터에서도 안정적 수렴을 보장하고, (2) 분류 헤드만 교체하면 인텐트 집합 확장(4→N 클래스)에도 쉽게 대응 가능하며, (3) HuggingFace Transformers 표준 인터페이스로 서비스 배포·모니터링·업그레이드 경로가 명확하다는 이점이 있다. 상세 선정 근거는 별도 산출물 『데이터 전처리 인공지능 학습 결과서』 1장을 참조한다.

3. 모델 학습 요약

3.1 데이터 구성

항목	값
학습 데이터 수	392건 (전체 490건의 80%)
검증 데이터 수	98건 (전체 490건의 20%)
평가(Test) 데이터 수	별도 확보 예정 (실사용 로그 기반 200~500건 계획)
분할 방식	Stratified Split (stratify=y, random_state=42)
데이터 경로	ai/intent/data/sample_data.csv

3.2 성능 평가 결과 (Validation 98건 기준)

지표	값
Accuracy	0.918
Precision (macro)	0.917
Recall (macro)	0.915
F1 Score (macro)	0.916

3.3 일반화 성능 평가

- **미검증 데이터 평가:** 실사용자 발화 로그 기반 독립 Test Set 수집 진행 중. 4/30 중간발표 이전까지 200건 이상 확보 및 성능 재측정 예정
- **과적합 방지:** Dropout(0.1) + Weight Decay(0.01) + Gradient Clipping(max_norm=1.0) + Linear Warmup + Validation Accuracy 기준 Best Model 저장
- **과소적합 여부:** 초기 Epoch부터 빠른 Loss 하락과 Val Accuracy 상승이 관찰되어 과소적합 징후 없음
- **Train/Val 간극:** Epoch 4 이후 소폭 확대되나 Val Accuracy는 0.90~0.95 구간에서 안정 유지 → 서비스 배포 허용 수준

4. 저장 및 배포

4.1 저장 파일 구성 (인텐트 분류 모델)

저장 위치: ai/intent/model/ (HuggingFace Transformers 표준 디렉터리 구조)

파일명	용도	크기	설명
model.safetensors	모델 가중치	약 431 MB	학습된 ElectraForSequenceClassification 가중치 (Safetensors 표준 포맷)
config.json	모델 구성	< 1 KB	Hidden/Heads/Layers 등 아키텍처 하이퍼파라미터 및 id2label 매핑
tokenizer.json	토큰나이저	약 797 KB	WordPiece vocab 및 정규화 규칙 (fast tokenizer)
tokenizer_config.json	토큰나이저 설정	< 1 KB	tokenizer 로드 시 필요한 메타 정 보 (model_max_length=512)
label_map.json	라벨 매핑	< 1 KB	인텐트 ID ↔ 레이블명 매핑 (0~3 → 장소추천/다이어리 작성/시설정 보/기타)
special_tokens_map.json	특수 토큰	< 1 KB	[CLS], [SEP], [PAD], [UNK], [MASK] 토큰 정의

파일 포맷: Safetensors (HuggingFace 표준 안전 포맷). pickle 기반 .bin 대비 역직렬화 시 임
의 코드 실행 위험이 없어 배포 환경에서 권장됨

4.2 입출력 명세

4.2.1 Input

항목	값	비고
자료형	str (UTF-8)	한국어 자연어 텍스트
최대 입력 길이	128 tokens	학습 시 truncation=True, max_length=128 기준 (tokenizer_config의 model_max_length=512는 상한이며 실사용은 128 고정)
Input Tensor Shape	(batch_size, sequence_length)	가변 길이 토큰 시퀀스 (padding='max_length' 시 128 고정)

항목	값	비고
필수 전처리	Tokenizer 적용	AutoTokenizer.from_pretrained(model_dir)의 결과물 사용

4.2.2 Output

항목	값	비고
Raw Logits Shape	(batch_size, 4)	분류 헤드 Linear 출력 (Softmax 전)
확률 분포	Softmax(logits)	각 클래스 확률, 합 = 1.0
최종 예측	argmax(probabilities)	정수 클래스 ID (0~3)
반환 정보	intent, confidence, probabilities	label_map.json을 통해 클래스 ID를 레이블명으로 변환하여 반환

4.2.3 클래스 목록 (label_map.json)

ID	레이블명	의미 및 후속 처리 체인
0	장소추천	조건 기반 장소 추천 → RAG 기반 하이브리드 검색 체인
1	다이어리 작성	6하원칙 기반 반려견 다이어리 생성 → 다이어리 체인
2	시설정보	특정 장소 상세·시설 정보 조회 → 장소 상세 체인
3	기타	인사·잡담·범위 밖 질의 → Fallback 응답 체인

4.3 모델 사양 요구 사항 (버전 및 환경)

항목	값
모델 버전	v1.0
Python 버전	>= 3.10 (Type Hint 문법 X Y 활용)
Deep Learning 프레임워크	PyTorch >= 2.0.0 (CUDA 11.8+ 호환)
Transformers	>= 4.35.0
추가 의존성	sentence-transformers (CLIP 임베딩용), chromadb (RAG), ultralytics (YOLOv8)
권장 하드웨어 (GPU)	NVIDIA GPU VRAM 4GB 이상 (T4, RTX 3060 이상 권장)
최소 하드웨어 (CPU)	x86_64 CPU 4 Core 이상, RAM 8GB 이상

항목	값
추론 소요시간 (GPU, 단건)	약 15~30 ms (batch=1, max_length=128)
추론 소요시간 (CPU, 단건)	약 150~300 ms (batch=1, max_length=128)
동시성	FastAPI + uvicorn worker 기준 GPU 1장당 약 100~200 RPS 처리 가능
OS	Linux (Ubuntu 22.04 검증), Windows/macOS 호환

requirements.txt 핵심 항목: torch>=2.0.0, transformers>=4.35.0, safetensors>=0.4.0, sentence-transformers>=2.2.0, chromadb>=0.4.0, ultralytics>=8.0.0, fastapi, uvicorn

4.4 모델 로드 및 추론 코드 — 인텐트 분류 (Python)

다음 코드는 ai/intent/evaluate.py 의 IntentClassifier 클래스로, 전처리(tokenization) → 추론 → 후처리(Softmax → argmax → label mapping) 전 과정을 캡슐화한다. 타입 힌트와 Google Style Docstring을 포함한다.

```
# ai/intent/evaluate.py - 인텐트 분류 모델 로드 및 추론
from __future__ import annotations

import json
from pathlib import Path

import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

MODEL_DIR = Path("ai/intent/model")
MAX_LENGTH = 128 # 학습 시 truncation 길이와 일치시켜야 성능 재현 가능

class IntentClassifier:
    """withDOG 인텐트 분류기.

    KoELECTRA-base-v3 Fine-tuned 모델을 로드하여
    한국어 발화를 4개 인텐트 중 하나로 분류한다.

    Attributes:
        tokenizer: ElectraTokenizer 인스턴스.
        model: ElectraForSequenceClassification 인스턴스.
        label_map: {class_id(int): label_name(str)} 매핑.
        device: 추론 장치 (cuda | cpu).
    """
```

```

def __init__(self, model_dir: Path = MODEL_DIR) -> None:
    """모델·토큰라이저·라벨 맵을 로드한다.

    Args:
        model_dir: safetensors 및 config 파일이 위치한 디렉터리.

    Raises:
        FileNotFoundError: 필수 파일이 누락된 경우.
    """
    self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    self.tokenizer = AutoTokenizer.from_pretrained(model_dir)
    self.model = AutoModelForSequenceClassification.from_pretrained(model_dir)
    self.model.to(self.device).eval()

    label_path = model_dir / "label_map.json"
    with label_path.open("r", encoding="utf-8") as f:
        raw_map: dict[str, str] = json.load(f)
    self.label_map = {int(k): v for k, v in raw_map.items()}

    @torch.inference_mode()
    def predict(self, text: str) -> dict[str, object]:
        """단일 발화의 인텐트를 예측한다.

        Args:
            text: 사용자 발화 원문.

        Returns:
            다음 키를 포함하는 딕셔너리:
            - intent (str): 예측 인텐트 레이블명
            - confidence (float): 최상위 클래스 확률 (0~1)
            - probabilities (dict[str, float]): 전 클래스 확률 분포

        Raises:
            ValueError: 입력이 빈 문자열이거나 None 인 경우.
        """
        if not text or not text.strip():
            raise ValueError("빈 문자열은 분류할 수 없습니다.")

        encoded = self.tokenizer(
            text,
            max_length=MAX_LENGTH,
            truncation=True,
            padding="max_length",
            return_tensors="pt",
        ).to(self.device)

        logits = self.model(**encoded).logits # shape: (1, 4)
        probs = torch.softmax(logits, dim=-1).squeeze(0) # shape: (4,)
        top_id = int(torch.argmax(probs).item())

        return {

```

```

        "intent": self.label_map[top_id],
        "confidence": float(probs[top_id].item()),
        "probabilities": {
            self.label_map[i]: float(probs[i].item())
            for i in range(len(self.label_map))
        },
    }

if __name__ == "__main__":
    classifier = IntentClassifier()
    sample = "강남역 근처 애견 동반 카페 알려줘"
    result = classifier.predict(sample)
    print(f"입력 : {sample}")
    print(f"예측 : {result['intent']} (신뢰도 {result['confidence']:.3f})")
    print(f"분포 : {result['probabilities']}")

```

4.5 모델 로드 및 추론 코드 — YOLOv8 Nano (Python)

YOLOv8 Nano는 Ultralytics 공식 체크포인트(yolov8n.pt, 약 6.3 MB)를 추론 전용으로 사용한다. COCO 사전학습 가중치의 'dog' 클래스(ID 16)만 필터링하여 강아지 객체 감지 결과를 반환한다.

```

# ai/vision/detect_breed.py - YOLOv8 Nano 로드 및 추론
from __future__ import annotations

from pathlib import Path

from ultralytics import YOLO
from ultralytics.engine.results import Results

YOLO_WEIGHTS = Path("yolov8n.pt")
DEFAULT_CONF = 0.25 # 낮은 신뢰도 감지 필터링 임계치

class DogDetector:
    """YOLOv8 Nano 기반 강아지 객체 감지기.

    Ultralytics 공식 체크포인트(yolov8n.pt)를 로드하여
    이미지 내 dog 카테고리 객체의 박스·신뢰도를 반환한다.

    Attributes:
        model: Ultralytics YOLO 인스턴스.
    """

    DOG_CLASS_ID = 16 # COCO dataset 'dog' class

    def __init__(self, weights_path: Path = YOLO_WEIGHTS) -> None:

```

```

"""가중치 파일을 로드한다.

Args:
    weights_path: .pt 체크포인트 경로.

Raises:
    FileNotFoundError: 가중치 파일이 존재하지 않는 경우.
"""
if not weights_path.exists():
    raise FileNotFoundError(f"YOLO 가중치 파일이 없습니다: {weights_path}")
self.model = YOLO(str(weights_path))

def detect(
    self,
    image_path: Path,
    conf_threshold: float = DEFAULT_CONF,
) -> list[dict[str, object]]:
    """이미지에서 강아지 객체를 감지한다.

    Args:
        image_path: 분석 대상 이미지 경로 (jpg/png 등).
        conf_threshold: 신뢰도 하한 (기본 0.25).

    Returns:
        감지된 강아지 객체 목록. 각 항목은 다음 키를 포함.
        - bbox (list[float]): [x1, y1, x2, y2] 픽셀 좌표
        - confidence (float): 감지 신뢰도
        - class_name (str): 'dog'

    Raises:
        FileNotFoundError: 입력 이미지가 존재하지 않는 경우.
"""
if not image_path.exists():
    raise FileNotFoundError(f"이미지 파일이 없습니다: {image_path}")

results: list[Results] = self.model.predict(
    source=str(image_path),
    conf=conf_threshold,
    classes=[self.DOG_CLASS_ID],
    verbose=False,
)

detections: list[dict[str, object]] = []
for result in results:
    if result.bboxes is None:
        continue
    for box in result.bboxes:
        detections.append({
            "bbox": box.xyxy[0].cpu().tolist(),
            "confidence": float(box.conf[0].item()),
            "class_name": result.names[int(box.cls[0].item())],
        })
return detections

```

```

if __name__ == "__main__":
    detector = DogDetector()
    sample_image = Path("ai/vision/samples/diary_001.jpg")
    dogs = detector.detect(sample_image)
    print(f"감지된 강아지 객체 수: {len(dogs)}")
    for i, dog in enumerate(dogs, start=1):
        print(f"[{i}] bbox={dog['bbox']}, confidence={dog['confidence']:.3f}")

```

4.6 모델 테스트

- **저장/로드 검증:** 동일 입력에 대한 반복 추론 시 확률값 편차 $\pm 1e-6$ 이내 (결정론적 수렴)
- **Inference 예시:** 입력 '강남역 근처 애견 동반 카페 알려줘' → 출력 장소추천 (신뢰도 0.95+)
- **Inference 예시:** 입력 '오늘 우리 강아지와 다녀온 공원 일기 써줘' → 출력 다이어리 작성 (신뢰도 0.90+)
- **배치 추론:** tokenizer(texts, padding=True, truncation=True, max_length=128)로 배치 입력 지원

4.7 배포 시 주의사항

- **max_length 고정:** tokenizer_config.json의 model_max_length는 512이나 학습은 128로 truncation되었으므로, 추론 시 반드시 max_length=128로 통일해야 학습 분포와 일치하는 성능을 재현할 수 있음
- **Safetensors 로드 선호:** .bin(pickle) 대신 .safetensors 파일을 사용하여 역직렬화 공격 위험 차단. transformers 4.35+ 부터 자동으로 safetensors 우선 로드
- **버전 고정:** transformers·torch·safetensors 버전 불일치 시 가중치 키 미스매치가 발생할 수 있음. requirements.txt에 == 또는 >= 지정 권장
- **label_map.json 동기화:** 라벨 ID 순서가 학습 시점과 다르면 추론 결과가 잘못 매핑됨. 모델 재학습 시 반드시 함께 갱신·커밋
- **device 관리:** model.to(device) 후 입력 텐서도 동일 device로 이동 필수. CPU/GPU 혼용 시 CUDA 오류 발생

- **torch.inference_mode 사용:** torch.no_grad보다 메모리·속도 측면에서 추가 최적화. 배포용 추론 코드에서 권장
- **YOLOv8 라이선스:** Ultralytics YOLOv8은 AGPL-3.0 라이선스이므로 상용 비공개 배포 시 라이선스 조건 확인 필요
- **COLD START:** 서비스 기동 시 첫 추론은 CUDA context 초기화로 수 초 지연됨. warm-up 호출 1회 실행 권장
- **GPU 메모리:** 모델 로드 후 VRAM 약 500 MB 점유. 타 서비스와 GPU 공유 시 oom 방지를 위해 torch.cuda.empty_cache() 주기적 호출

5. 종합 평가 및 활용 방안

5.1 모델 안정성

Validation 98건에 대해 저장·재로드 반복 시 예측 레이블과 확률값의 편차가 $\pm 1e-6$ 이내로 관찰되어, 배포 환경에서 결정론적 추론이 보장된다. Safetensors 포맷은 pickle 기반 역직렬화 위험 없이 해시 검증 기반 로드가 가능하므로 무결성 측면에서도 안전하다.

5.2 일반화 가능성

클래스별 F1-score 편차가 0.86~0.93 범위로 제한되어 특정 클래스에 과도하게 편향된 학습이 발생하지 않았다. 다만 FACILITY_INFO(시설정보) 클래스는 PLACE_RECOMMEND(장소추천)와 의미 경계가 모호하여 상대적으로 낮은 F1을 보이며, 이는 실사용 Test Set 수집 후 재평가가 필요한 포인트이다.

5.3 재사용성

- **저장 용량:** 약 432 MB (가중치 431 MB + 토큰나이저 0.8 MB + 기타 메타 < 10 KB)
- **추론 평균 시간:** GPU 기준 약 0.02초/건, CPU 기준 약 0.2초/건 (batch=1, max_length=128)
- **분류 헤드 교체 용이성:** ElectraForSequenceClassification 인터페이스를 유지하면 num_labels만 변경하여 인텐트 수 확장 가능

5.4 향후 활용 방안

- **API 서버 탑재:** FastAPI /api/intent/predict 엔드포인트로 노출, Docker 컨테이너 배포
- **챗봇 전처리 라우터:** LLM 호출 이전 단계에서 인텐트 분기하여 LLM 호출 비용 절감 및 응답 품질 안정화
- **오답 로그 기반 재학습:** 서비스 로그에서 수집된 오답 샘플을 주간 단위로 재학습 세트에 편입 (MLOps 루프 연계)
- **유사 질문 추천:** 인텐트 단위로 FAQ 그룹핑 후 유사 질의 추천 UX에 활용

6. 추가 기재

6.1 저장된 모델 파일 위치 / URL

- **인텐트 분류 모델 디렉터리:** ai/intent/model/ (프로젝트 로컬 경로)
- **YOLOv8 Nano 가중치:** /yolov8n.pt (프로젝트 루트, 약 6.3 MB)
- **원본 체크포인트 (Base):** <https://huggingface.co/monologg/koelectra-base-v3-discriminator>
- **YOLOv8 공식 저장소:** <https://github.com/ultralytics/ultralytics>
- **Fine-tuned 모델 릴리스:** GitHub Release에 업로드 예정 (v1.0 태그)

6.2 저장/불러오기 코드 스니펫

학습 완료 후 모델을 저장하는 코드(참고용).

```
# ai/intent/train.py (발체) - 학습 후 모델 저장
from pathlib import Path

OUTPUT_DIR = Path("ai/intent/model")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# Safetensors 형식으로 모델 가중치·config·토큰라이저 일괄 저장
model.save_pretrained(OUTPUT_DIR, safe_serialization=True)
tokenizer.save_pretrained(OUTPUT_DIR)

# 라벨 맵은 별도 JSON 으로 관리 (학습 스크립트 외부에서 쉽게 참조 가능)
import json
label_map = {0: "장소추천", 1: "다이어리 작성", 2: "시설정보", 3: "기타"}
with (OUTPUT_DIR / "label_map.json").open("w", encoding="utf-8") as f:
```

```
json.dump({str(k): v for k, v in label_map.items()}, f, ensure_ascii=False,  
indent=2)
```

6.3 학습 로그

- **실험 추적 도구:** Weights & Biases (wandb) 또는 TensorBoard — 수집 진행 중
- **첨부 예정 자료:** Train/Val Loss 곡선, Val Accuracy 곡선, Confusion Matrix, Classification Report 원문 캡처
- **관련 산출물:** 『데이터 전처리 인공지능 학습 결과서 v2』(4장 성능 평가, 5장 과적합 대응) 참조